

Despliegue Automatizado y Administración de Servicios en VPS

Aplicación: SPA To-Do List con CI/CD (GitHub Actions + Nginx + Node.js + MySQL)

Tamara Alejandra Chuez Sánchez

Período académico:

2026-1

Materia:

Administración de Data Center

UNIVERSIDAD ESTATAL PENÍNSULA DE SANTA ELENA

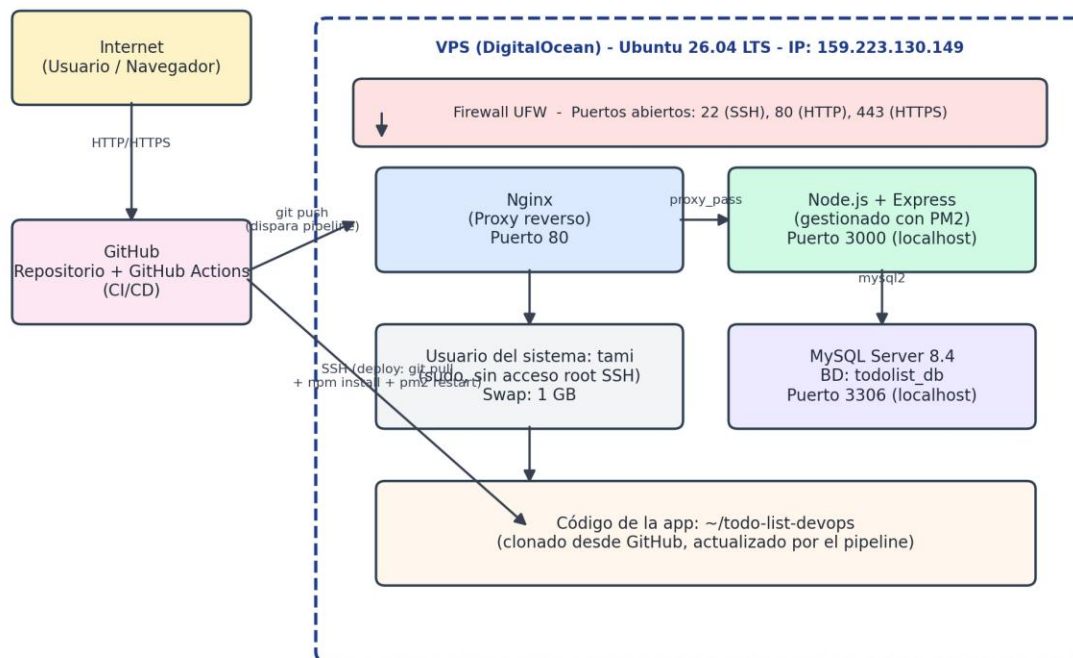
Introducción

El presente informe documenta el desarrollo del Trabajo Práctico "Despliegue Automatizado y Administración de Servicios en VPS", correspondiente a la asignatura de DevOps. El objetivo del proyecto fue asumir el rol de administrador de sistemas para gestionar el ciclo de vida completo de una aplicación web tipo To-Do List (SPA), desde el aprovisionamiento de un servidor virtual privado (VPS) hasta la automatización de despliegues mediante un pipeline de Integración y Despliegue Continuo (CI/CD) con GitHub Actions.

El stack tecnológico implementado consta de: Nginx como servidor web y proxy reverso, Node.js con el framework Express como servidor de aplicaciones, y MySQL como motor de base de datos. Todo el proceso fue documentado paso a paso, incluyendo los inconvenientes encontrados durante la implementación y su respectiva resolución.

1. Diseño de la Infraestructura

A continuación se presenta el diagrama de red de la infraestructura implementada, junto con la descripción de cada uno de sus componentes.



1.1 Descripción de los componentes

Componente	Función	Detalle técnico
VPS (DigitalOcean)	Servidor virtual que aloja toda la infraestructura	Ubuntu 26.04 LTS "Resolute Raccoon" · 1 vCPU · 512 MB RAM + 1 GB swap · 10 GB disco · IP pública 159.223.130.149
Nginx	Servidor web / proxy reverso	Escucha en el puerto 80 y redirige las peticiones al backend Node.js en el puerto 3000 (proxy_pass)
Node.js + Express	Servidor de aplicaciones (API REST)	Puerto 3000, accesible solo desde localhost. Gestionado con PM2 para persistencia y reinicio automático
MySQL 8.4 LTS	Motor de base de datos relacional	Puerto 3306, accesible solo desde localhost. Base de datos: todolist_db, tabla: tasks
GitHub	Repositorio de código y orquestador CI/CD	Aloja el código fuente y ejecuta el workflow de GitHub Actions ante cada push a la rama main
UFW (firewall)	Control de acceso a nivel de red	Solo permite tráfico entrante en los puertos 22 (SSH), 80 (HTTP) y 443 (HTTPS)

1.2 Justificación del diseño

- Se separaron las responsabilidades siguiendo el patrón clásico de 3 capas: Web (Nginx) - Aplicación (Node/Express) - Datos (MySQL), todas dentro del mismo VPS por tratarse de un entorno académico de bajo tráfico.
- El backend y la base de datos no exponen sus puertos (3000 y 3306) directamente a internet; solo son accesibles desde el propio servidor (localhost), reduciendo la superficie de ataque.
- Se optó por un plan de VPS económico (512 MB RAM) complementado con un archivo swap de 1 GB, para mantener estable el consumo de memoria del stack completo bajo un presupuesto académico.

2. Proceso de Provisionamiento

Esta sección constituye la bitácora de instalación y configuración de los servicios del VPS, en el orden en que fueron ejecutados.

2.1 Datos generales del servidor

Parámetro	Valor
Proveedor	DigitalOcean
Sistema operativo	Ubuntu 26.04 LTS (Resolute Raccoon)
Dirección IP pública	159.223.130.149
Usuario administrador	tami (grupo sudo)
Autenticación SSH	Por llave pública Ed25519 (login por contraseña deshabilitado)
Acceso root por SSH	Deshabilitado (PermitRootLogin no)

2.2 Bitácora de configuración (hardening inicial)

- Actualización completa del sistema operativo: `apt update && apt upgrade -y` (incluyó actualización de kernel de 7.0.0-27 a 7.0.0-28, con reinicio del servidor).
- Creación de usuario administrador no-root: `adduser tami`, con permisos `sudo` mediante `usermod -aG sudo tami`.
- Copia de la llave SSH del usuario al nuevo usuario para autenticación sin contraseña (`rsync --archive --chown=tami:tami ~/.ssh /home/tami`).
- Configuración de archivo swap de 1 GB (`fallocate`, `mkswap`, `swapon`) para compensar la memoria RAM limitada (512 MB) del plan contratado.
- Configuración del firewall UFW, permitiendo únicamente los puertos necesarios: OpenSSH (22), HTTP (80) y HTTPS (443); todo el resto del tráfico entrante es bloqueado por defecto.
- Deshabilitación del acceso SSH directo como root (`PermitRootLogin no` en `/etc/ssh/sshd_config`), obligando a usar el usuario `tami` con `sudo` para tareas administrativas.

2.3 Puertos abiertos (firewall UFW)

Puerto	Protocolo	Servicio	Alcance
22	TCP	SSH	Acceso remoto administrativo (llave pública únicamente)
80	TCP	HTTP (Nginx)	Tráfico web público, redirigido internamente al backend
443	TCP	HTTPS (reservado)	Preparado para certificado SSL/TLS futuro
3000	TCP	Node.js / Express	Solo accesible desde localhost (no expuesto a internet)
3306	TCP	MySQL	Solo accesible desde localhost (no expuesto a internet)

2.4 Instalación del stack

- Nginx: `sudo apt install nginx -y` — verificado con `systemctl status nginx` (active/running).
- Node.js v24.18.0 (LTS) y npm 11.16.0: instalados mediante el repositorio oficial de NodeSource.
- MySQL Server 8.4.10 (LTS, incluido por defecto en Ubuntu 26.04): `sudo apt install mysql-server -y`, asegurado con `mysql_secure_installation`.
- PM2 (gestor de procesos para Node.js): instalado globalmente con npm, configurado con `pm2 startup` y `pm2 save` para que la aplicación se reinicie automáticamente ante un reinicio del servidor.

2.5 Configuración de la base de datos

```
CREATE DATABASE todolist_db;
CREATE USER 'todo_app'@'localhost' IDENTIFIED BY '*****';
GRANT ALL PRIVILEGES ON todolist_db.* TO 'todo_app'@'localhost';
FLUSH PRIVILEGES;

USE todolist_db;
CREATE TABLE tasks (
  id INT AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(255) NOT NULL,
  completed BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Se creó un usuario de base de datos dedicado (`todo_app`) con privilegios limitados exclusivamente a la base `todolist_db`, en lugar de usar la cuenta `root` de MySQL desde la aplicación, siguiendo el principio de mínimo privilegio.

3. Configuración del Pipeline CI/CD

El pipeline fue implementado con GitHub Actions y automatiza el despliegue de la aplicación cada vez que se realiza un push a la rama `main` del repositorio.

3.1 Flujo del pipeline

- Disparador (trigger): el workflow se ejecuta automáticamente ante cualquier push a la rama `main`.
- Paso 1 — Checkout: la acción `actions/checkout@v4` descarga el código del repositorio dentro del entorno temporal de ejecución de GitHub (runner).
- Paso 2 — Conexión SSH: mediante la acción `appleboy/ssh-action`, GitHub Actions se conecta de forma segura al VPS utilizando credenciales almacenadas como GitHub Secrets (nunca expuestas en el código).
- Paso 3 — Actualización del código en el servidor: dentro del VPS se ejecuta `git pull origin main` para traer los últimos cambios.

- Paso 4 — Instalación de dependencias: `npm install --production` reinstala únicamente las dependencias necesarias en producción.
- Paso 5 — Reinicio de la aplicación: `pm2 restart todo-app` aplica los cambios sin downtime perceptible, gracias a la gestión de procesos de PM2.

3.2 Gestión segura de credenciales (GitHub Secrets)

Las credenciales de acceso al VPS nunca se almacenan en el código fuente. En su lugar, se configuraron 4 GitHub Secrets a nivel de repositorio:

Secret	Contenido
VPS_HOST	Dirección IP pública del VPS
VPS_USER	Usuario SSH no-root (tami)
VPS_SSH_KEY	Llave privada SSH dedicada exclusivamente al pipeline (Ed25519), distinta a la llave personal del administrador
VPS_PORT	Puerto SSH (22)

Como buena práctica de seguridad, se generó una llave SSH exclusiva para GitHub Actions (`github_actions_deploy`), separada de la llave personal utilizada para la administración manual del servidor. Esto permite revocar el acceso del pipeline de forma independiente si fuera necesario, sin afectar el acceso administrativo normal.

3.3 Archivo de definición del workflow

```
name: Deploy to VPS

on:
  push:
    branches:
      - main

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Deploy via SSH
        uses: appleboy/ssh-action@v1.0.3
        with:
          host: ${ secrets.VPS_HOST }
          username: ${ secrets.VPS_USER }
          key: ${ secrets.VPS_SSH_KEY }
          port: ${ secrets.VPS_PORT }
          script: |
            cd ~/todo-list-devops
            git pull origin main
```

```
cd backend
npm install --production
pm2 restart todo-app
```

3.4 Incidencia detectada durante las pruebas del pipeline

Durante la validación del pipeline se presentaron fallos consecutivos de tipo "Startup failure" en todas las ejecuciones del workflow, incluso en una prueba mínima sin relación con SSH. Tras descartar errores de sintaxis YAML, problemas de permisos del repositorio y el nombre del repositorio (que inicialmente contenía un guion al principio, corregido como buena práctica), se confirmó mediante el panel de estado oficial de GitHub (githubstatus.com) que existía un incidente activo del proveedor afectando a GitHub Actions a nivel global en el momento de las pruebas ("Incident with GitHub Actions", 19-20 de julio de 2026). Este hallazgo evidencia la importancia de verificar el estado de los servicios externos como parte del diagnóstico de fallos en un pipeline CI/CD, antes de asumir un error de configuración propio.

3.5 Confirmación del despliegue automático exitoso

Una vez resuelto el incidente por parte de GitHub, se validó el pipeline completo de extremo a extremo mediante una prueba real: se modificó un texto visible en el frontend de la aplicación (public/index.html), y se realizó un push a la rama main, sin realizar ningún cambio manual en el VPS.

Evidencia	Resultado
Commit de prueba	f12f866 — "Probar despliegue automatico end-to-end"
Workflow ejecutado	Deploy to VPS #2
Resultado de la ejecución	Successful — "All checks have passed / 1 successful check: Deploy to VPS / deploy (push) Successful in 15s"
Verificación en producción	El cambio de texto ("Mi Lista de Tareas - CI/CD OK") se reflejó correctamente en http://159.223.130.149 sin intervención manual en el servidor

Con esta prueba se confirma que el pipeline CI/CD funciona correctamente de extremo a extremo: un cambio de código subido a GitHub se despliega automáticamente en el VPS de producción (git pull + npm install + pm2 restart) en aproximadamente 15 segundos, cumpliendo con el requerimiento central del trabajo práctico.

4. Plan de Mantenimiento y Seguridad

4.1 Estrategia de respaldo de la base de datos

Para asegurar que los datos almacenados en MySQL no se pierdan ante una falla del servidor, se define la siguiente estrategia de respaldo basada en mysqldump y una tarea programada (cron):

```
#!/bin/bash
# /home/tami/backup_db.sh
FECHA=$(date +%Y-%m-%d_%H-%M)
mysqldump -u todo_app -p'*****' todolist_db >
```

```
/home/tami/backups/todolist_db_$FECHA.sql
find /home/tami/backups -name "*.sql" -mtime +7 -delete
```

Esta tarea se programa para ejecutarse automáticamente todos los días mediante crontab (crontab -e):

```
0 3 * * * /home/tami/backup_db.sh
```

- Frecuencia: respaldo diario a las 3:00 a.m. (horario de bajo tráfico).
- Retención: se conservan los últimos 7 días de respaldos; los más antiguos se eliminan automáticamente para no saturar el disco (10 GB disponibles en el plan contratado).
- Recomendación de mejora futura: copiar los archivos .sql generados hacia un almacenamiento externo (por ejemplo, DigitalOcean Spaces o Google Drive vía rclone), para no depender únicamente del disco del propio VPS ante una falla total del servidor.

4.2 Políticas de firewall aplicadas

Regla	Descripción
Política por defecto	Denegar todo el tráfico entrante que no esté explícitamente permitido (deny incoming)
Puerto 22 (SSH)	Permitido — único método de acceso administrativo; autenticación exclusivamente por llave pública
Puerto 80 (HTTP)	Permitido — tráfico público hacia la aplicación web a través de Nginx
Puerto 443 (HTTPS)	Permitido — reservado para una futura implementación de certificado SSL/TLS (ej. Let's Encrypt)
Resto de puertos	Bloqueados por defecto, incluyendo 3000 (Node.js) y 3306 (MySQL), que solo son accesibles internamente (localhost)

4.3 Otras medidas de seguridad implementadas

- Autenticación SSH exclusivamente por llave pública Ed25519; el acceso por contraseña está deshabilitado.
- Acceso SSH como root deshabilitado (PermitRootLogin no); toda tarea administrativa se realiza con el usuario tami mediante sudo.
- Separación de credenciales: llave SSH personal (uso administrativo manual) distinta de la llave SSH del pipeline de GitHub Actions (uso exclusivo del CI/CD).
- Usuario de base de datos con privilegios limitados a una sola base de datos (todo_app), en lugar de usar la cuenta root de MySQL desde la aplicación.
- Variables sensibles (credenciales de base de datos) almacenadas en un archivo .env en el servidor, excluido del control de versiones mediante .gitignore.

4.4 Plan de mantenimiento recurrente

Tarea	Frecuencia
Actualización de paquetes del sistema (apt update && apt upgrade)	Mensual
Respaldo de la base de datos (mysqldump automatizado)	Diario
Revisión de logs de Nginx y de la aplicación (PM2 logs)	Semanal
Revisión del uso de disco y memoria (df -h, free -h)	Semanal
Rotación / revisión de llaves SSH	Semestral

Conclusiones

El desarrollo de este trabajo práctico permitió aplicar de forma integral los conceptos de administración de sistemas y metodologías DevOps: aprovisionamiento de infraestructura en la nube, hardening de seguridad de un servidor Linux, despliegue de un stack web completo (Nginx, Node.js, MySQL) y automatización del ciclo de vida de una aplicación mediante un pipeline de Integración y Despliegue Continuo mostrando cómo un cambio en el código, al subirse a la rama principal del repositorio, se refleja automáticamente en el servidor de producción sin intervención manual.

Asimismo, el proceso evidenció la importancia de diagnosticar sistemáticamente los fallos en un entorno de producción real, diferenciando entre errores de configuración propios e incidencias externas del proveedor de servicios, y reforzó buenas prácticas de seguridad como la separación de privilegios, el principio de mínimo acceso, y la gestión segura de credenciales mediante secretos cifrados.